Заголовочные файлы

По мере увеличения размера программ весь код уже не помещается в нескольких файлах, записывать каждый раз **предварительные объявления** для функций, которые мы хотим использовать, но которые находятся в других файлах, становится всё утомительное и утомительное. Хорошо было бы, если бы все предварительные объявления находились в одном месте, не так ли?

Файлы .cpp не являются единственными файлами в проектах. Есть еще один тип файлов — **заголовочные файлы** (или ***«заголовки»***), которые имеют расширение `.h`. Целью заголовочных файлов является удобное хранение набора объявлений объектов для их последующего использования в других программах.

Оглавление:

1. **Заголовочные файлы из Стандартной библиотеки C++**
2. **Пишем свои собственные заголовочные файлы**
3. **Угловые скобки (<>) vs. Двойные кавычки ("")**
4. **Почему iostream пишется без окончания .h?**
5. **Можно ли записывать определения в заголовочных файлах?**
6. **Советы**

Заголовочные файлы из Стандартной библиотеки C++

Рассмотрим следующую программу:

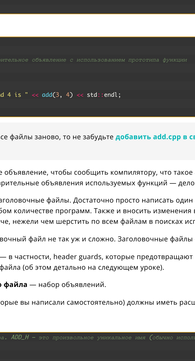
```
#include <iostream>

int main()
{
    std::cout << "Hello, world!" << std::endl;
    return 0;
}
```

Результат выполнения программы:

```
Hello, world!
```

В этой программе мы используем **cout**, который нигде не определяем. Откуда компилятор знает, что это такое? Дело в том, что cout объявлен в заголовочном файле iostream. Когда мы пишем `#include <iostream>`, мы делаем запрос, чтобы всё содержимое заголовочного файла iostream было скопировано в наш файл. Таким образом, всё содержимое библиотеки iostream становится доступным для использования.

Как правило, в заголовочных файлах записываются только объявления, без определений. Следовательно, если cout только объявлен в заголовочном файле iostream, то где же он определяется? Ответ: в Стандартной библиотеке C++, которая автоматически подключается к вашему проекту на этапе линкинга.



Подумайте о последствиях отсутствия заголовочного файла iostream. Каждый раз, при использовании cout, вам бы приходилось вручную копировать все предварительные объявления, связанные с cout в верхнюю часть вашего файла! Хорошо ведь, что можно просто указать `#include <iostream>`, не так ли?

Пишем свои собственные заголовочные файлы

Теперь давайте вернемся к примеру, который мы обсуждали на предыдущем уроке. У нас было два файла: add.cpp и main.cpp.

add.cpp:

```
int add(int x, int y)
{
    return x + y;
}
```

main.cpp:

```
#include <iostream>

int add(int x, int y); // предварительное объявление с использованием прототипа функции

int main()
{
    std::cout << "The sum of 3 and 4 is " << add(3, 4) << std::endl;
    return 0;
}
```

> ***Примечание:*** Если вы создаете все файлы заново, то не забудьте **добавить add.cpp в свой проект**, чтобы он был подключен к компиляции.

Мы использовали предварительное объявление, чтобы сообщить компилятору, что такое add(). Как мы уже говорили, записывать в каждом файле предварительные объявления используемых функций — дело не слишком увлекательное.

И здесь нам на помощь приходят заголовочные файлы. Достаточно просто написать один заголовочный файл и его можно будет повторно использовать в любом количестве программ. Также и вносить изменения в такой код (например, добавление еще одного параметра) гораздо легче, нежели чем шерстить по всем файлам в поисках используемых функций.

Написать свой собственный заголовочный файл не так уж и сложно. Заголовочные файлы состоят из двух частей:

- **Директивы препроцессора** — в частности, header guards, которые предотвращают вызов заголовочного файла больше одного раза из одного и того же файла (об этом детально на следующем уроке).
- **Содержимое заголовочного файла** — набор объявлений.

Все ваши заголовочные файлы (которые вы написали самостоятельно) должны иметь расширение `.h`.

add.h:

```
// Начало и директив препроцессора. ADD_H - это произвольное уникальное имя (обычно используется имя заголовочного файла)
#ifndef ADD_H
#define ADD_H

// А это уже содержимое заголовочного файла
int add(int x, int y); // прототип функции add() (не забывайте точку с запятой в конце!)

// Заканчиваем директивой препроцессора
#endif
```

Чтобы использовать этот файл в main.cpp, вам сначала нужно будет подключить его к проекту.

main.cpp, в котором мы подключаем add.h:

```
#include <iostream>
#include "add.h"

int main()
{
    std::cout << "The sum of 3 and 4 is " << add(3, 4) << std::endl;
    return 0;
}
```

add.cpp остается без изменений:

```
int add(int x, int y)
{
    return x + y;
}
```

Когда компилятор встречает `#include "add.h"`, он копирует всё содержимое add.h в текущий файл. Таким образом, мы получаем предварительное объявление функции add().

> ***Примечание***: При подключении заголовочного файла, всё его содержимое вставляется сразу же после строки `#include ....`

Если вы получили ошибку от компилятора, что add.h не найден, то убедитесь, что имя вашего файла точно «add.h». Вполне возможно, что вы могли сделать опечатку, например, просто «add» (без «.h») или «add.h.txt» или «add.hpp».

Если вы получили ошибку от линкера, что функция add() не определена, то убедитесь, что вы корректно подключили add.cpp к вашему проекту (и к компиляции тоже)!

Угловые скобки (<>) vs. Двойные кавычки ("")

Вы, наверное, хотите узнать, почему используются угловые скобки для iostream и двойные кавычки для add.h. Дело в том, что, используя угловые скобки, мы сообщаем компилятору, что подключаемый заголовочный файл написан не нами (он является «системным», т.е. предоставляется Стандартной библиотекой C++), так что искать этот заголовочный файл следует в системных директориях. Двойные кавычки сообщают компилятору, что мы подключаем наш собственный заголовочный файл, который мы написали самостоятельно, поэтому искать его следует в текущей директории нашего проекта. Если файла там не окажется, то компилятор начнет проверять другие пути, в том числе и системные директории.

Правило: Используйте угловые скобки для подключения «системных» заголовочных файлов и двойные кавычки для ваших заголовочных файлов.

Стоит отметить, что одни заголовочные файлы могут подключать другие заголовочные файлы. Тем не менее, так делать не рекомендуется.

Почему iostream пишется без окончания .h?

Еще один часто задаваемый вопрос: «Почему iostream (или любой другой из стандартных заголовочных файлов) при подключении пишется без окончания «.h»?». Дело в том, что есть 2 отдельных файла: iostream.h (заголовочный файл) и просто iostream! Для объяснения потребуется краткий экскурс в историю.

Когда C++ только создавался, все файлы библиотеки Runtime имели окончание .h. Оригинальные версии cout и cin объявлены в iostream.h. При стандартизации языка C++ комитетом ANSI, решили перенести все функции из библиотеки Runtime в пространство имен std, чтобы предотвратить возможность возникновения конфликтов имен с пользовательскими идентификаторами (что, между прочим, является хорошей идеей). Тем не менее, возникла проблема: если все функции переместить в пространство имен std, то старые программы перестали работать!

Для обеспечения обратной совместимости ввели новый набор заголовочных файлов с теми же именами, но без окончания «.h». Весь их функционал находится в пространстве имен std. Таким образом, старые программы с `#include <iostream.h>` не нужно было переписывать, а новые программы уже могли использовать `#include <iostream>`.

Когда вы подключаете заголовочный файл из Стандартной библиотеки C++, убедитесь, что вы используете версию без .h (если она существует). В противном случае, вы будете использовать устаревшую версию заголовочного файла, который уже больше не поддерживается.

Кроме того, многие библиотеки, унаследованные от языка Си, которые до сих пор используются в C++, также были продублированы с добавлением префикса `c` (например, stdlib.h стал **c**stdlib). Функционал этих библиотек также перенесли в пространство имен std, чтобы избежать возможность возникновения конфликтов имен с пользовательскими идентификаторами.

Правило: При подключении заголовочных файлов из Стандартной библиотеки C++, используйте версию без «.h» (если она существует). Пользовательские заголовочные файлы должны иметь окончание «.h».

Можно ли записывать определения в заголовочных файлах?

Язык C++ не будет жаловаться, если вы это сделаете, но так делать не принято.

Как уже было сказано выше, при подключении заголовочного файла, всё его содержимое вставляется сразу же после строки с #include. Это означает, что любые определения, которые есть в заголовочном файле, скопируются в ваш файл.

Для небольших проектов, это, скорее всего, не будет проблемой. Но для более крупных это может способствовать увеличению времени компиляции (так как код будет повторно компилироваться) и размеру исполняемого файла. Если внести изменения в определения, которые находятся в файле .cpp, то перекомпилировать придется только этот файл. Если же внести изменения в определения, которые записаны в заголовочном файле, то перекомпилировать придется каждый файл, который подключает этот заголовок, используя директиву препроцессора #include. И вероятность того, что из-за одного небольшого изменения вам придется перекомпилировать весь проект, резко возрастает!

Иногда делаются исключения для простых функций, которые вряд ли изменятся (например, где определение состоит всего лишь из одной строки).

Советы

Вот несколько советов по написанию собственных заголовочных файлов:

- Всегда используйте директивы препроцессора.
- Не определяйте переменные в заголовочных файлах, если это не константы. Заголовочные файлы следует использовать только для объявлений.
- Не определяйте функции в заголовочных файлах.
- Каждый заголовочный файл должен выполнять свое конкретное задание и быть как можно более независимым. Например, вы можете поместить все ваши объявления, связанные с файлом A.cpp в файл A.h, а все ваши объявления, связанные с B.cpp — в файл B.h. Таким образом, если вы будете работать только с A.cpp, то вам будет достаточно подключить только A.h и наоборот.
- Используйте имена ваших рабочих файлов в качестве имен для ваших заголовочных файлов (например, grades.h работает с grades.cpp).
- Не подключайте одни заголовочные файлы из других заголовочных файлов.
- Не подключайте файлы .cpp, используя директиву препроцессора #include.